

DOSSIER TECHNIQUE

iCity GED

Technologies, architecture et bibliothèques

Logiciel de gestion des dossiers de marchés publics
iCity — branche Smart City d'ABA Technology

Auteur	Ichrak Elhantati
Date	24 septembre 2026
Dépôt	github.com/elhantatiichraq16-web/icity-ged-app

Le projet en bref

iCity réalise des marchés publics au Maroc : vidéosurveillance, contrôle d'accès, alarme, détection incendie, maintenance. Chaque marché produit une pile de pièces — contrat, ordre de service, bons de livraison, procès-verbaux de réception, attestations, mainlevée de caution — qu'il faut retrouver vite et dont il faut savoir ce qui manque.

iCity GED (gestion électronique des documents) est le logiciel qui range ces pièces, les lit, les rattache à leur marché et suit chaque affaire jusqu'à sa clôture. Il remplace une première version montée sur **Paperless-ngx** (sous Docker), complétée d'un portail Python et d'une vingtaine de scripts : trop générique pour des marchés publics, trop lourde pour le poste, et éclatée entre plusieurs outils.

14 000 lignes de code	71 routes d'API	18 tables	19 écrans	≈ 200 tests automatisés	194 documents réels
---------------------------------	---------------------------	---------------------	---------------------	-----------------------------------	-------------------------------

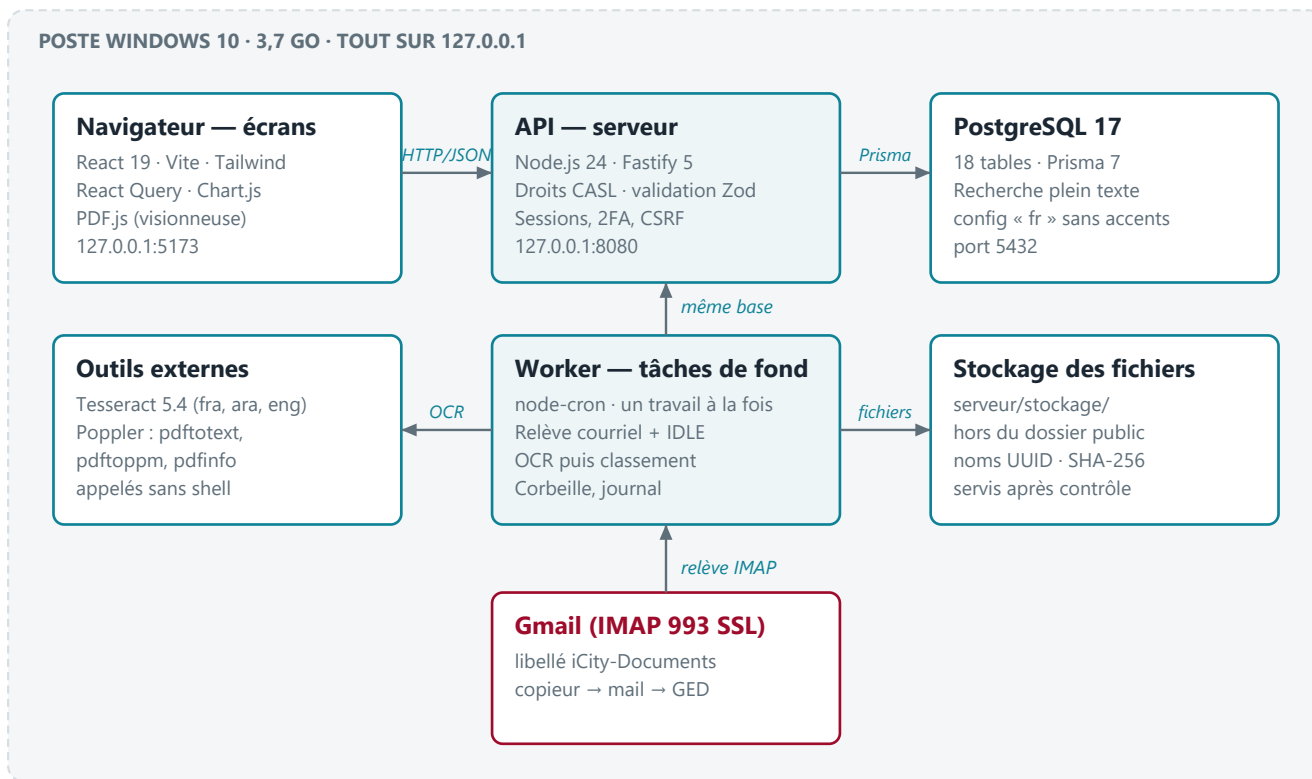
Les contraintes qui ont guidé les choix

- **Un poste Windows 10 de 3,7 Go de mémoire.** Pas de Docker, pas d'Elasticsearch : la recherche est confiée à la base, et la lecture des scans (OCR) traite **un document à la fois**.
- **Tout reste en local.** L'application n'écoute que sur 127.0.0.1 : elle n'est jamais visible depuis le réseau, et les marchés ne quittent pas le poste.
- **Une seule application,** un seul langage (JavaScript) du navigateur jusqu'au serveur, pour ne plus jongler entre plusieurs outils.
- **Aucun secret dans le code.** Mots de passe et clés vivent dans serveur/.env, qui ne part jamais dans Git.

Un choix assumé. Le cahier des charges initial envisageait Laravel (PHP). Le projet a été construit en **JavaScript de bout en bout** — React pour les écrans, Node.js pour le serveur — ce qui permet de partager les mêmes règles (droits, validation des formulaires, calcul des phases) entre le navigateur et le serveur, sans les écrire deux fois. La base, d'abord MySQL, a ensuite été migrée vers **PostgreSQL 17** (764 lignes transférées sans perte).

Architecture

L'application tient en **trois processus** lancés ensemble par `npm run dev`, une base de données et un dossier de fichiers. Tout tourne sur le même poste.



COMPOSANT	RÔLE
Écrans (client)	Ce que l'utilisateur voit dans son navigateur. Une application React « d'une seule page » : on change d'écran sans recharger. Elle ne parle qu'à l'API, jamais directement à la base.
API (serveur)	Le cœur : reçoit chaque demande, vérifie qui la fait et s'il en a le droit, valide les données, lit ou écrit en base, et renvoie du JSON.
Worker	Un second processus Node.js qui travaille en fond : relève le courriel, lit les scans (OCR), classe les pièces lues, vide la corbeille. Séparé de l'API pour qu'un OCR de 80 pages ne ralentisse jamais les écrans.
PostgreSQL	La base : marchés, documents, texte lu, utilisateurs, courriels, journal d'audit.
Stockage	Les fichiers eux-mêmes (PDF, images) restent sur le disque, hors de la base : la base garde leur chemin et leur empreinte. Une sauvegarde doit donc prendre les deux .

Organisation du code

Le dépôt est un **monorepo** : un seul dossier Git qui contient trois paquets, reliés par les *workspaces* de npm. Le paquet commun est importé à la fois par le serveur et par le client.

```

icity-ged/
├─ commun/      règles partagées navigateur ↔ serveur
│  └─ src/      roles.js · droits.js · marches.js · circuit.js · schemas.js
├─ serveur/     l'API, le worker, la base
│  └─ prisma/   schema.prisma (18 tables) · migrations · seed.js
│     └─ src/
│        └─ app.js, index.js  démarrage de Fastify, sécurité
│        └─ worker.js         tâches de fond
│        └─ routes/           11 fichiers, un par domaine (marchés, documents...)
│        └─ services/         19 fichiers : les règles métier
│        └─ plugins/          authentication
│  └─ scripts/               installer · sauvegarder · restaurer · importer
│  └─ tests/                 tests automatisés (Vitest)
└─ client/                 les écrans
   └─ src/
      └─ pages/              un fichier par écran
      └─ coquille/           barre latérale, recherche Ctrl+K, thème
      └─ ui/                 composants : boutons, champs, visionneuse...

```

Le découpage routes / services

Côté serveur, une **route** ne fait que recevoir la demande, vérifier les droits et répondre. Le travail réel est dans les **services** : `phase-marche.js` déduit la phase d'un marché, `ocr.js` lit les scans, `classement.js` reconnaît référence, client et type dans le texte, `courriel-imap.js` relève la boîte Gmail. Ce découpage permet de tester une règle métier sans lancer de serveur.

Conventions

- Le code métier est **en français** : `Marche`, `Document`, `Client`, `verserFichier()`.
- Les commentaires expliquent **pourquoi** une règle existe, pas ce que fait la ligne suivante.
- La base évolue par **migrations** Prisma : chaque changement de structure est un fichier SQL daté, rejoué à l'identique sur chaque poste.

Technologies utilisées

Vue par couche. Les versions sont celles installées dans le projet.

COUCHE	TECHNOLOGIE	POURQUOI CE CHOIX
Langage	JavaScript (ES modules)	Un seul langage partout ; les règles de commun/ servent au navigateur et au serveur.
Exécution serveur	Node.js 24	Lit le .env sans bibliothèque, relance tout seul en développement (--watch).
Serveur HTTP	Fastify 5	Plus léger et plus rapide qu'Express, avec un système de plugins propre (sécurité, cookies, fichiers).
Base de données	PostgreSQL 17	Recherche plein texte intégrée, en français et sans accents : pas besoin d'Elasticsearch.
Accès à la base	Prisma 7	Le schéma décrit en un fichier ; requêtes typées ; migrations versionnées.
Interface	React 19	Écrans composés de composants réutilisables ; mise à jour sans recharger la page.
Outil de build	Vite 8	Démarrage en 2 secondes, rechargement instantané pendant le développement.
Styles	Tailwind CSS 4	Le design se fait dans le composant, avec les couleurs d'iCity et un mode sombre.
Lecture des scans	Tesseract 5.4	OCR libre, langues français, arabe et anglais ; lancé seulement quand le PDF n'a pas de texte.
Outils PDF	Poppler	pdftotext extrait le texte existant, pdftoppm transforme les pages en images pour l'OCR.
Courriel	IMAP / SMTP	Relève de Gmail (libellé dédié) et envoi des réponses depuis l'application.
Tests	Vitest 5	Tests rapides, sur une base de test séparée qui ne touche jamais la vraie.
Versions	Git + GitHub	Historique complet ; dépôt icity-ged-app.

Les bibliothèques

Chaque dépendance a un rôle précis. Le projet évite les bibliothèques lourdes : le poste a peu de mémoire, et chaque dépendance est du code qu'on ne maîtrise pas.

Serveur — serveur/package.json

BIBLIOTHÈQUE	VERSION	À QUOI ELLE SERT
fastify	5.12	Le serveur HTTP de l'API.
@fastify/helmet	13.1	En-têtes de sécurité : Content-Security-Policy, X-Frame-Options, nosniff...
@fastify/cookie	11.1	Lecture et écriture du cookie de session.
@fastify/rate-limit	11.2	Limite les tentatives de connexion (contre les essais de mots de passe en rafale).
@fastify/multipart	10.1	Réception des fichiers versés par glisser-déposer.
@fastify/static	10.1	Sert les écrans construits, en production.
fastify-plugin	6.0	Écrit le plugin d'authentification partagé par toutes les routes.
@prisma/client + adapter-pg	7.10	Accès à PostgreSQL depuis le code.
prisma (outil)	7.10	Génère le client et applique les migrations.
@casl/ability	7.0	Les droits : qui peut lire, verser, valider, supprimer — selon le rôle et la confidentialité.
zod	4.6	Valide chaque donnée reçue (formulaires, paramètres) et renvoie des messages en français.
bcryptjs	3.0	Hache les mots de passe (coût 12) ; version pur JavaScript, sans compilation sous Windows.
otplib + qrcode	13.5 / 1.5	Double authentification : codes à 6 chiffres et QR code pour l'application mobile.
imapflow	2.0	Relève la boîte Gmail en IMAP, et l'écoute en continu (IDLE).
mailparser	3.9	Découpe un mail : expéditeur, objet, corps, pièces jointes.
nodemailer	10.0	Envoie les mails : invitation, mot de passe oublié, réponses aux clients.
sanitize-html	2.17	Nettoie le HTML des mails reçus avant de l'afficher.
file-type	22.1	Vérifie le vrai type d'un fichier d'après son contenu, pas son extension.
node-cron	4.6	Planifie les tâches du worker (relève toutes les 10 min, entretien chaque nuit).

Écrans — client/package.json

BIBLIOTHÈQUE	VERSION	À QUOI ELLE SERT
react + react-dom	19.3	Construire l'interface en composants.
react-router	8.4	Passer d'un écran à l'autre (URL /marches, /documents...).
@tanstack/react-query	5.103	Charger les données de l'API, les garder en cache et les rafraîchir.
radix-ui	1.6	Composants accessibles au clavier et aux lecteurs d'écran : modales, menus.
tailwindcss	4.3	Les styles, le mode sombre, le responsive.
chart.js + react-chartjs-2	4.5 / 5.3	Les graphiques du tableau de bord.

BIBLIOTHÈQUE	VERSION	À QUOI ELLE SERT
pdfjs-dist	6.3	La visionneuse PDF intégrée (PDF.js de Mozilla).
lucide-react	1.47	Les icônes.
@fontsource/*	5.3	Les polices Fraunces, Inter Tight et JetBrains Mono, embarquées (aucun appel à Internet).
vite + plugin-react	8.3 / 6.1	Serveur de développement et construction des écrans.

Partagé et outillage

BIBLIOTHÈQUE	VERSION	À QUOI ELLE SERT
zod , @casl/ability (commun)	4.6 / 7.0	Les mêmes schémas de validation et les mêmes droits, côté écran et côté serveur.
vitest	5.0	Les tests automatisés.
concurrently	10.0	Lance l'API, les écrans et le worker d'une seule commande.

La base de données

18 tables, décrites dans `serveur/prisma/schema.prisma`. Les modèles portent des noms français ; les tables, des noms en minuscules.

DOMAINE	TABLES	CE QU'ELLES CONTIENNENT
Identités	Role , Utilisateur , Session , Jeton , Journal	7 rôles, les comptes, les connexions ouvertes, les liens d'invitation, et le journal d'audit (qui, quoi, quand, avant/après).
Référentiels	Client , TypeDocument , Etiquette	Les maîtres d'ouvrage et leurs synonymes, les 21 types de pièces, les 22 étiquettes.
Marchés et pièces	Marche , Document , DocumentEtiquette , DocumentVersion , Suggestion , Doublon	Les affaires, les documents avec leur texte lu, leurs versions successives, les rapprochements de doublons.
Courriel	CompteMail , Releve , Mail , PieceJointeMail	Le compte Gmail (mot de passe chiffré), l'historique des relèves, les messages et leurs pièces jointes.

Trois choix de conception

- **La phase d'un marché n'est jamais saisie.** Elle se calcule à partir des pièces présentes : un ordre de service → « En exécution », un PV provisoire → « Réception provisoire », une mainlevée → « Clôturé ».
- **La référence normalisée est unique.** « 23C/2017/TGR » et ses variantes mal lues par l'OCR retombent sur la même clé : deux lectures d'une même affaire ne créent pas deux marchés.
- **La recherche plein texte** utilise une configuration PostgreSQL nommée `fr` qui enchaîne unaccent et le radical français : « vidéosurveillance » trouve « vidéosurveillance », et « marchés » trouve « marché ».

Comment ça marche

Le parcours d'une pièce, de son arrivée à la recherche.



- **Empreinte.** Le serveur calcule le SHA-256 du fichier. S'il existe déjà au fonds, le versement est refusé et l'original est montré.
- **Lecture.** Un PDF qui porte déjà son texte est lu par pdftotext, sans OCR. Un scan passe en file d'attente : le worker le découpe en images (pdftoppm, 300 dpi) et Tesseract le lit page par page, **un document à la fois**.
- **Classement.** Le texte lu est analysé : la référence de marché la plus citée, le client reconnu (motifs, synonymes, domaines e-mail), le type de pièce. La machine **ne remplit que les cases vides** et renonce dans le doute : une pièce non classée se voit, une pièce mal classée se perd. Chaque décision est écrite au journal d'audit.

Le courriel

Le worker relève le libellé Gmail iCity-Documents toutes les 10 minutes, et l'écoute en continu (IMAP IDLE) quand c'est possible. Il ne se fie pas au « non lu » : il pose le libellé iCity-Verse sur chaque message traité et mémorise son identifiant, pour ne jamais le traiter deux fois. Chaque pièce jointe devient un document, rattaché au client reconnu par le domaine de l'expéditeur. Un copieur réglé en « scan vers e-mail » fait arriver un papier dans la GED en quelques minutes.

Les tâches de fond

TÂCHE	QUAND	CE QU'ELLE FAIT
Relève du courriel	toutes les 10 min + IDLE	Verse les pièces jointes des nouveaux messages.
File OCR	toutes les 2 min	Lit les scans en attente, un par un, puis les classe.
Corbeille	chaque nuit, 3 h	Supprime définitivement ce qui y est depuis plus de 30 jours.
Journal d'audit	chaque nuit, 3 h	Purge les entrées de plus de deux ans.
Sauvegarde	dimanche, 2 h	Base (pg_dump) et fichiers, avec rotation.

Sécurité

Les marchés publics contiennent des montants, des cautions et des pièces confidentielles. La sécurité a été prise en compte dès la première phase, pas ajoutée à la fin.

MENACE	PROTECTION
Vol de mot de passe	Hachage bcrypt (coût 12) ; limite de tentatives ; double authentification optionnelle (TOTP + codes de secours).
Vol de session	Le cookie ne contient qu'un jeton aléatoire ; la base n'en garde que l'empreinte SHA-256. Session régénérée à la connexion, expiration.
Requête forgée (CSRF)	Un jeton CSRF par session, exigé sur toute action qui modifie.
Accès non autorisé	Droits CASL sur chaque route : 7 rôles, 4 niveaux de confidentialité. Un lecteur ne voit pas un document « Confidentiel », même dans la recherche.
Fichier piégé	Type vérifié sur le contenu réel ; nom remplacé par un UUID ; stockage hors du dossier public ; servi seulement après contrôle des droits.
Injection de commande	Tesseract et Poppler sont appelés avec un tableau d'arguments, jamais une chaîne de commande.
Mail malveillant	HTML nettoyé par sanitize-html ; aucune donnée externe (objet, nom de fichier, texte OCR) injectée comme HTML.
Fuite des secrets	Mot de passe Gmail et secret 2FA chiffrés en base (AES-256-GCM) ; tous les secrets dans .env, hors de Git.
Exposition réseau	API et écrans sur 127.0.0.1 uniquement ; en-têtes CSP, X-Frame-Options, nosniff, Referrer-Policy.
Perte de données	Corbeille de 30 jours, versions jamais écrasées, sauvegarde base + fichiers avec restauration testée.

Qualité, tests et lancement

Les tests

Environ **200 tests automatisés** (Vitest), dans `commun/tests` et `serveur/tests`. Ils tournent sur une base `icity_ged_test` séparée : le script de préparation refuse toute base dont le nom ne finit pas par `_test`, pour ne jamais vider la vraie par erreur. Ils couvrent la normalisation des références, le calcul des phases, le classement, les droits par rôle, la confidentialité, le courriel, la corbeille et la sauvegarde.

Les commandes

COMMANDE	CE QU'ELLE FAIT
<code>npm install</code>	Installe toutes les bibliothèques des trois paquets.
<code>npm run installer -w serveur</code>	Crée le rôle et les bases PostgreSQL, écrit le <code>.env</code> avec des secrets tirés au hasard.
<code>npm run db:migrer</code>	Crée ou met à jour les tables.
<code>npm run db:seed</code>	Installe les rôles, types, étiquettes, clients et le compte administrateur.
<code>npm run dev</code>	Lance l'API (8080), les écrans (5173) et le worker.
<code>npm test</code>	Lance tous les tests.
<code>npm run sauvegarder -w serveur</code>	Sauvegarde complète, horodatée.
<code>npm run restaurer -w serveur</code>	Vérifie une sauvegarde ; avec <code>--oui</code> , la restaure.

Prérequis sur le poste

- Node.js 22 ou plus (installé : 24.14)
- PostgreSQL 17, qui démarre avec Windows
- Poppler (`pdftotext`, `pdftoppm`, `pdftinfo`)
- Tesseract 5 avec les langues français, arabe et anglais — nécessaire seulement pour les scans sans texte

En résumé. iCity GED est une application web locale en JavaScript : React et Vite pour les écrans, Node.js et Fastify pour l'API, PostgreSQL et Prisma pour les données, Tesseract et Poppler pour la lecture des documents. Elle tient sur un poste modeste, garde tout en local, et chaque règle métier est couverte par des tests.